

Київський національний університет імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Кафедра інтелектуальних програмних систем

ЗВІТ

до лабораторної роботи №1

Тема: Tiger Language, регулярні вирази та детермінізація автомата

Виконав студент 3-го курсу:

Міцкевич Костянтин Олександрович

Київ – 2026

1. Мета роботи

Метою лабораторної роботи було ознайомлення з базовими можливостями навчальної мови Tiger, виконання простих програм з використанням функцій, рекурсії та обробки введення, побудова регулярних виразів для заданих мов, а також аналіз і детермінізація недетермінованого скінченного автомата.

2. Ініціалізація лабораторної роботи

Для виконання лабораторної роботи було використано середовище Ubuntu 24. На початку роботи було створено окрему директорію lab1 та імпортовано skeleton-проект dragon-tiger.

Після імпорту проекту було перевірено, що в каталозі наявні готовий бінарний файл src/driver/dtiger та runtime-бібліотека src/runtime/posix/libruntime.a.

Під час першої спроби лінування на Ubuntu виникла помилка, пов'язана з режимом PIE. Для її усунення під час лінування було використано прапорець -no-pie. Це дозволило коректно зібрати виконувані файли для всіх програм, написаних мовою Tiger.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab1$ wget -qO- www.sifflez.org/lectures/compil/lab1/dragon-tiger.tar.gz | tar xzv
dragon-tiger/
dragon-tiger/Makefile
dragon-tiger/src/
dragon-tiger/src/driver/
dragon-tiger/src/driver/.gitignore
dragon-tiger/src/driver/dtiger
dragon-tiger/src/runtime/
dragon-tiger/src/runtime/posix/
dragon-tiger/src/runtime/posix/libruntime.a
dragon-tiger/src/runtime/posix/.gitignore
```

3. Початковий аналіз структури проекту

Після імпорту проекту було проаналізовано його структуру. Основна робота в межах першої лабораторної роботи виконувалась не шляхом модифікації самого компілятора, а шляхом створення нових вихідних файлів мовою Tiger та текстових файлів з регулярними виразами.

Ключовими елементами skeleton-проекту були: src/driver/dtiger — готовий компілятор Tiger; src/runtime/posix/libruntime.a — runtime-бібліотека, необхідна для лінування згенерованих об'єктних файлів; а також створені в межах

лабораторної роботи файли `hello.tig`, `fibonacci.tig`, `read_unsigned.tig`, `regex1.txt` та `regex2.txt`.

Таким чином, у цій роботі використовувався вже підготовлений інструментарій, а основний акцент було зроблено на реалізації програм мовою Tiger, формальному описі мов регулярними виразами та аналізі автомата.

Файл	Призначення
<code>hello.tig</code>	перевірка базового компілювання та виведення тексту
<code>fibonacci.tig</code>	демонстрація функцій, рекурсії, умовних виразів і циклу <code>for</code>
<code>read_unsigned.tig</code>	обробка введення, перевірка символів і формування числа
<code>regex1.txt</code>	опис мови, де перша <code>a</code> передуює першій <code>b</code>
<code>regex2.txt</code>	опис мови з парною кількістю символів <code>a</code>

4. Реалізація лабораторної роботи

4.1. Реалізація програми `hello.tig`

Першим завданням було створення найпростішої програми `hello.tig`, що виводить рядок `Hello World!` Це завдання є базовою перевіркою того, що компілятор `dtiger` коректно трансліює програму мовою Tiger в об'єктний файл, а `runtime`-бібліотека успішно підключається на етапі лінування.

Зміст файла `hello.tig` зводиться до одного виклику функції `print` з рядком `Hello World!` та символом нового рядка. Попри простоту, цей приклад є важливим, оскільки дозволяє перевірити весь мінімальний ланцюжок: написання програми, її компіляцію, лінування та запуск.

Для перевірки виконання використовувались команди:

```
src/driver/dtiger -o hello.o hello.tig
```

```
clang++ -no-pie -g hello.o src/runtime/posix/libruntime.a -o hello
```

```
./hello
```

Результатом виконання програми став коректний вивід рядка Hello World!, що підтвердило працездатність інструментарію та правильність мінімальної Tiger-програми.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ src/driver/dtiger -o hello.o hello.tig
clang++ -no-pie -g hello.o src/runtime/posix/libruntime.a -o hello
./hello
Hello World!
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$
```

4.2. Реалізація програми fibonacci.tig

Другим завданням було створення програми fibonacci.tig, яка обчислює значення функції Фібоначчі рекурсивно та виводить кілька перших елементів послідовності. Це завдання має значно більшу практичну цінність, ніж попереднє, оскільки дозволяє перевірити роботу оголошення функцій, рекурсивних викликів, умовних виразів if та циклу for у мові Tiger.

У програмі було реалізовано функцію fibonacci, для якої базові випадки $F(0)=1$ та $F(1)=1$ повертаються без рекурсії, а для більших значень використовується рекурентне співвідношення. Далі в головному блоці програми цикл for послідовно обчислює і друкує значення функції для аргументів від 1 до 8.

Таке завдання показує, що мова Tiger підтримує не лише простий вивід, а й повноцінну функціональну логіку з рекурсією. Крім того, воно дозволяє побачити, як у Tiger поєднуються різні мовні конструкції в межах однієї програми.

```
src/driver/dtiger -o fibonacci.o fibonacci.tig
clang++ -no-pie -g fibonacci.o src/runtime/posix/libruntime.a -o fibonacci
./fibonacci
```

У результаті програма вивела послідовність значень: 1, 2, 3, 5, 8, 13, 21, 34. Це підтвердило правильність реалізації рекурсії та умовної логіки.

```
netto world:
• user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ src/driver/dtiger -o fibonacci.o fibonacci.tig
clang++ -no-pie -g fibonacci.o src/runtime/posix/libruntime.a -o fibonacci
./fibonacci
1
2
3
5
8
13
21
34
```

4.3. Реалізація програми read_unsigned.tig

Найбільш змістовним завданням Тігер-частини стало створення програми read_unsigned.tig. Її метою було зчитування рядка зі стандартного вводу, перевірка, чи складається він лише з цифр, та перетворення цього рядка на беззнакове ціле число. Якщо введення містить нецифрові символи або є порожнім, функція повинна повертати -1.

Зміст цього завдання полягає в тому, щоб продемонструвати вміння працювати з примітивами getchar() та ord(), виконувати посимвольну обробку введення, будувати десяткове число вручну та реалізовувати перевірку коректності введених даних. На відміну від попередніх завдань, тут важливо не лише отримати результат, а й правильно обробити помилкові сценарії.

У реалізації було використано рекурсивний підхід. Допоміжна логіка накопичує поточне значення числа у змінній ass, перевіряє кожен символ на належність до діапазону від '0' до '9', а також фіксує, чи було прочитано хоча б одну цифру. Якщо зустрічається некоректний символ, програма повертає -1. У головному блоці далі виводиться значення a + 2, що відповідає умові завдання.

```
src/driver/dtiger -o read_unsigned.o read_unsigned.tig
clang++ -no-pie -g read_unsigned.o src/runtime/posix/libruntime.a -o
read_unsigned
echo "123" | ./read_unsigned
echo "0" | ./read_unsigned
echo "12a" | ./read_unsigned
echo "" | ./read_unsigned
```

Було перевірено як коректні, так і помилкові сценарії. Для вводу 123 програма вивела 125, для 0 — значення 2, а для некоректного вводу 12a та порожнього рядка результатом стало 1, оскільки функція `read_unsigned()` повертала -1, після чого головний блок друкував $a + 2$.

```
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ src/driver/dtiger -o read_unsigned.o read_unsigned.tig
clang++ -no-pie -g read_unsigned.o src/runtime/posix/libruntime.a -o read_unsigned
echo "123" | ./read_unsigned
echo "0" | ./read_unsigned
echo "12a" | ./read_unsigned
echo "" | ./read_unsigned
125
2
1
1
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$
```

4.4. Реалізація регулярних виразів

У наступній частині лабораторної роботи потрібно було побудувати два регулярні вирази для мов над алфавітом $\{a, b\}$. На відміну від попередніх підзавдань, тут робота виконувалась уже не в Tiger, а засобами формального опису мов.

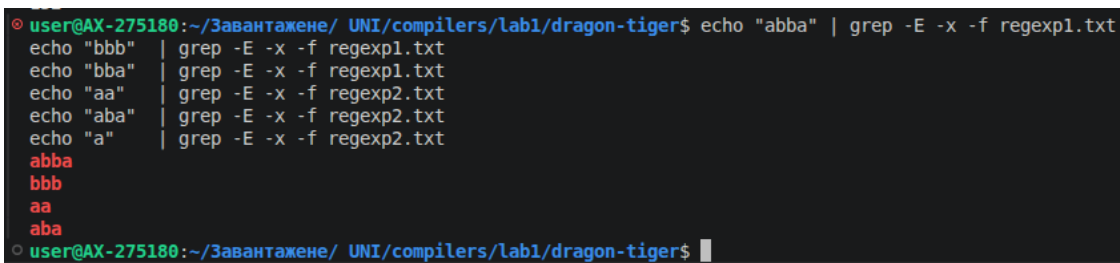
Для файла `regex1.txt` потрібно було побудувати вираз для всіх слів, у яких перша літера a , якщо вона існує, передує першій літері b , якщо вона існує. Зміст цього завдання полягає в правильному формулюванні умови мовою регулярних виразів. У підсумку було використано вираз $^(b^*|a[ab]^*)\$$, який допускає або лише символи b , або слова, що починаються з a , після чого можуть містити довільну комбінацію a та b .

Для файла `regex2.txt` потрібно було описати мову всіх слів із парною кількістю символів a . Сенс цього завдання полягає в тому, щоб навчитися описувати не окремі конкретні слова, а структурну властивість мови. Було використано вираз $^b^*(ab^*ab^*)^*\$$, який представляє будь-яку кількість символів b до, між та після пар символів a .

Перевірка регулярних виразів виконувалась за допомогою `grep -E`. Для `regex1.txt` було підтверджено, що підходять слова `abba`, `bbb` та `a`, але не

підходять bba і ba. Для regexp2.txt було підтверджено, що підходять aa, abba, aba та baa, а слово a не задовольняє умову.

```
echo "abba" | grep -E -x -f regexp1.txt
echo "bbb" | grep -E -x -f regexp1.txt
echo "bba" | grep -E -x -f regexp1.txt
echo "aa" | grep -E -x -f regexp2.txt
echo "aba" | grep -E -x -f regexp2.txt
echo "a" | grep -E -x -f regexp2.txt
```



```
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "abba" | grep -E -x -f regexp1.txt
abba
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "bbb" | grep -E -x -f regexp1.txt
bbb
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "bba" | grep -E -x -f regexp1.txt
bba
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "aa" | grep -E -x -f regexp2.txt
aa
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "aba" | grep -E -x -f regexp2.txt
aba
user@AX-275180:~/Завантажене/ UNI/compilers/lab1/dragon-tiger$ echo "a" | grep -E -x -f regexp2.txt
```

4.5. Аналіз і детермінізація автомата

Остання частина лабораторної роботи була присвячена аналізу недетермінованого автомата з ϵ -переходами. Початковий автомат містить переходи за символами x, y, z та декілька ϵ -переходів між станами 1, 2, 3 і 4, а також між станами 5, 6 і 7. Через це один і той самий вхідний рядок може розглядатися через кілька шляхів, а отже автомат не є детермінованим.

Після аналізу переходів було встановлено, що автомат приймає мову $(xz)^*(x|y)$. Це означає, що допускаються слова x і y, а також будь-яка кількість повторень блоку xz перед завершальним символом x або y. Наприклад, автомат приймає x, y, xzx, xzy, xzxzx та xzxzy.

Автомат не є детермінованим, тому що в ньому є ϵ -переходи, тобто переходи без зчитування символу. Для детермінованого автомата так бути не повинно: у ньому для кожного стану і кожного символу має бути один чіткий наступний перехід. Через це наш автомат треба перетворити у детермінований.

Спочатку беремо стартовий стан 1 і дивимось, куди з нього можна дійти тільки по ϵ -переходах. Так ми потрапляємо ще в стани 2, 3 і 4, тому перший стан нового автомата буде $\{1,2,3,4\}$.

Далі дивимось, що буде, якщо з цього стану читати символи:

- по x зі стану 1 можна перейти в 5, а звідти по ϵ — у 6 і 7, тому отримуємо стан $\{5,6,7\}$;
- по y зі стану 2 можна перейти в 6, а звідти по ϵ — у 7, тому отримуємо стан $\{6,7\}$;
- по z з множини $\{1,2,3,4\}$ перейти нікуди не можна, тому тут маємо порожню множину $\emptyset$.

Потім треба розглянути вже нові стани, які з'явилися:

- зі стану $\{5,6,7\}$ по z можна перейти до стану 2, а з нього по ϵ знову дійти до $\{1,2,3,4\}$. По x і y переходів уже немає, тому там буде $\emptyset$;
- зі стану $\{6,7\}$ уже не можна перейти ні в який корисний новий стан, тому всі переходи з нього теж ведуть у $\emptyset$.

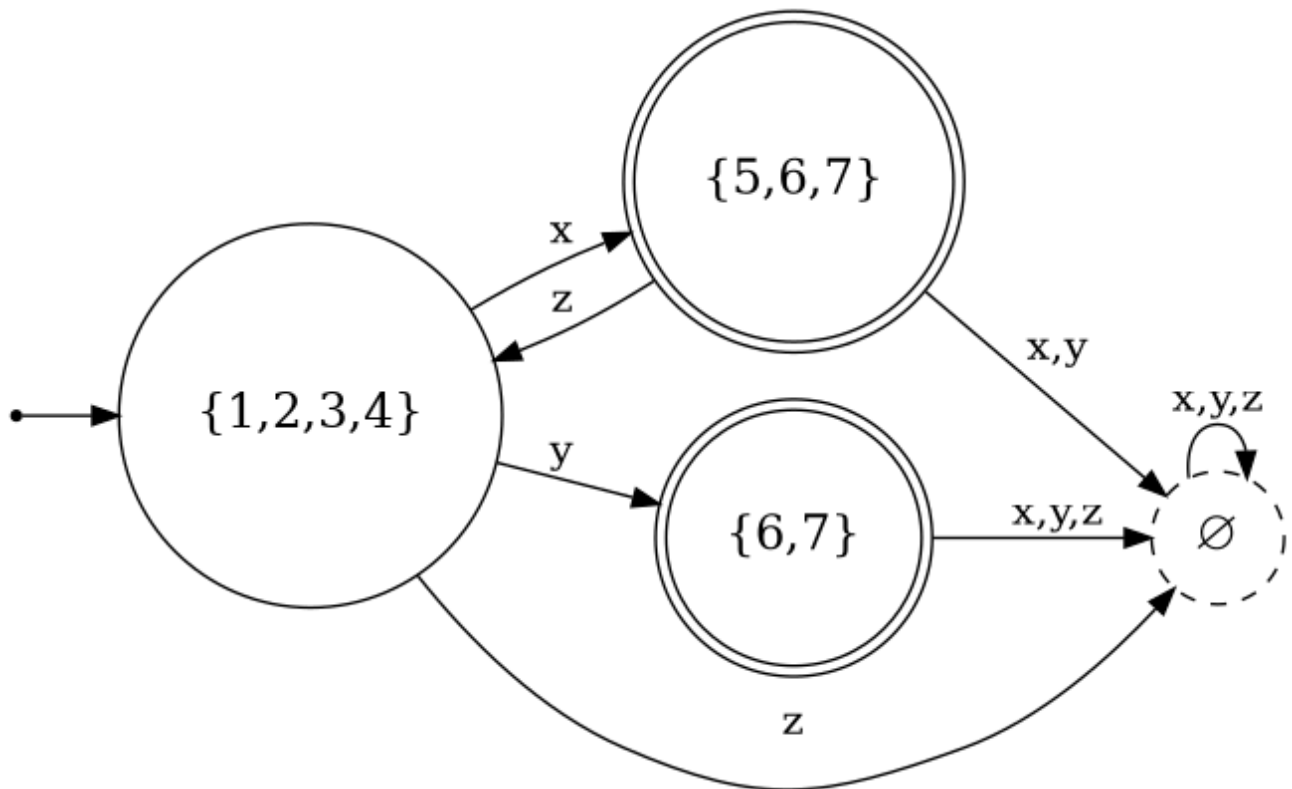
Отже, після детермінізації отримуємо такі стани:

- $A = \{1,2,3,4\}$ — стартовий;
- $B = \{5,6,7\}$ — приймальний;
- $C = \{6,7\}$ — приймальний;
- $D = \emptyset$ — мертвий стан.

Стани B і C є приймальними, тому що в них є стан 7, а саме він був кінцевим у початковому автоматі.

Стан	x	y	z
$A = \{1,2,3,4\}$	B	C	D
$B = \{5,6,7\}$	D	D	A
$C = \{6,7\}$	D	D	D
$D = \emptyset$	D	D	D

Таким чином, детермінізація дозволила замінити початковий недетермінований автомат компактним DFA з чотирма станами, який приймає ту саму мову.



5. Висновок

У ході виконання лабораторної роботи було успішно реалізовано три програми мовою Tiger, побудовано два регулярні вирази для заданих мов та виконано аналіз недетермінованого автомата з подальшою детермінізацією. Під час роботи було перевірено використання функцій `print`, `print_int`, `getchar` та `ord`, реалізовано рекурсію, цикл `for`, перевірку введення та формальний опис мов регулярними виразами.

Особливу увагу було приділено детермінізації автомата, оскільки саме ця частина вимагає чіткого переходу від інтуїтивного аналізу шляхів до формального методу побудови DFA через множини станів. У результаті було отримано еквівалентний детермінований автомат з чотирма станами, що приймає мову $(xz)^*(x|y)$.